

Informe

MAKERLAB EIA

Valentina Díaz Oviedo

María Camila Galán

Samuel Munevar González

Santiago Orrego Orrego

Programación Orientada a Objetos

Universidad EIA

2026-2

Grupo 03

Johan Gabriel Vélez

Contenido

1. Alcance del Proyecto	4
2. Requisitos Funcionales	5
Registrar Usuarios:	5
Diferenciar los tipos de Usuarios:	5
Registrar Laboratorios:.....	5
Registrar Equipos:	5
Manejar diferentes Equipos:.....	5
Registrar Capacitaciones y Certificaciones:.....	5
Validar certificaciones:	5
Crear Reservas:.....	5
Validar la Disponibilidad de una Reserva:.....	5
Abrir y Cerrar Sesiones de Uso:.....	5
3. Requisitos No Funcionales.....	6
Interfaz:	6
Mantenibilidad	6
Coherencia entre Modelo e Implementación:.....	6
4. Reglas de Negocio	7
Identificadores únicos:	7
Horario válido:.....	7
Certificación vigente:	7
Certificación correspondiente:.....	7
No superposición de reservas:	7
Estado operativo del equipo:	7
Control de sesiones:	7
Mantenimiento trazable:	7
5. Riesgos del Proyecto	8
Inconsistencia entre UML y código:	8
Validaciones incompletas:.....	8
Errores de integración entre clases:	8
6. Decisiones de Modelado	9
Herencias:.....	9
Formación:	9

Reservas y Uso:	9
Mantenimiento:	9
Polimorfismo:	9
7. Conclusiones.....	10

1. Alcance del Proyecto

El proyecto MakerLab EIA tiene como propósito diseñar e implementar un sistema orientado a objetos que permita administrar el uso de laboratorios y equipos especializados.

El sistema toma en cuenta la gestión de diferentes tipos de usuarios, incluyendo estudiantes, profesores, técnicos y auxiliares de laboratorio, así como el riesgo de laboratorios, equipos, capacitaciones, certificaciones, reservas, sesiones de uso, listas de espera, tickets de mantenimiento y notificaciones.

El sistema permite representar el ciclo básico de utilización de un equipo: un usuario puede contar con una capacitación y certificación asociada a un tipo de equipo, realizar una reserva, utilizar el equipo mediante una sesión de uso y registrar posteriormente información como consumibles, observaciones e incidentes. También se contempla la intervención de técnicos en procesos de mantenimiento y el cambio de estado operativo de los equipos.

El alcance de la primera entrega está centrado en los conceptos de programación orientada a objetos trabajados durante las clases, especialmente clases, objetos, atributos, métodos, herencias, clases abstractas, polimorfismo entre otros. El proyecto se desarrolla como una aplicación de consola en Java y mantiene los datos en memoria, de acuerdo con las restricciones establecidas para esta primera entrega.

2. Requisitos Funcionales

Los requisitos funcionales describen las principales funciones que el sistema debe permitir realizar.

Registrar Usuarios: El sistema debe permitir registrar usuarios mediante un identificador único, nombre, correo electrónico y tipo de usuario.

Diferenciar los tipos de Usuarios: El sistema debe permitir manejar diferentes tipos de usuarios con comportamientos particulares, incluyendo estudiantes, profesores y técnicos. Adicionalmente, el modelo contempla auxiliares de laboratorio.

Registrar Laboratorios: El sistema debe permitir manejar diferentes tipos de usuarios con comportamientos particulares, incluyendo estudiantes, profesores y técnicos. Adicionalmente, el modelo contempla auxiliares de laboratorio.

Registrar Equipos: El sistema debe permitir registrar equipos indicando código, nombre, nivel de riesgo, estado y laboratorio al que pertenecen.

Manejar diferentes Equipos: El sistema debe representar diferentes tipos específicos de equipo mediante especialización de la clase **Equipos**. Actualmente se contemplan (Impresora 3D, Cortadora Láser y Multímetro)

Registrar Capacitaciones y Certificaciones: El sistema debe permitir asociar procesos de formación y certificación a los usuarios y a los equipos correspondientes. La clase **Formacion** sirve como base para **Capacitaciones** y **Certificacion**.

Validar certificaciones: El sistema debe permitir verificar si una certificación se encuentra vigente antes de permitir el uso de un equipo que requiera autorización.

Crear Reservas: El sistema debe permitir crear reservas indicando usuario, equipo, fecha, hora de inicio, hora de finalización, propósito y estado.

Validar la Disponibilidad de una Reserva: El sistema debe comprobar que el laboratorio se encuentre disponible durante la franja solicitada y contemplar las condiciones necesarias para realizar una reserva.

Abrir y Cerrar Sesiones de Uso: El sistema debe permitir iniciar una sesión asociada a una reserva y posteriormente cerrarla registrando la hora de finalización, consumibles utilizados, observaciones e incidentes.

3. Requisitos No Funcionales

Interfaz: El sistema debe ejecutarse mediante consola, sin requerir una interfaz gráfica para cumplir con el alcance de la Entrega 1.

Mantenibilidad: El código debe estar organizado mediante clases con responsabilidades diferenciadas, aplicando principios de encapsulamiento, herencia, abstracción y polimorfismo.

Coherencia entre Modelo e Implementación: El diagrama UML debe mantener correspondencia con las clases, atributos, métodos, relaciones y tipos utilizados en la implementación Java. Estas restricciones corresponden al alcance tecnológico definido por el proyecto, que establece Java 26, Maven, ejecución por consola y almacenamiento de datos en memoria.

4. Reglas de Negocio

Identificadores únicos: Los identificadores de usuarios, laboratorios, equipos, reservas, certificaciones y tickets no deben repetirse.

Horario válido: La hora de finalización de una reserva debe ser posterior a la hora de inicio y la reserva debe encontrarse dentro del horario de funcionamiento del laboratorio.

Certificación vigente: Un usuario no puede reservar un equipo que requiera certificación si la certificación se encuentra vencida.

Certificación correspondiente: La certificación del usuario debe corresponder al tipo de equipo que desea utilizar.

No superposición de reservas: Dos reservas confirmadas para el mismo equipo no pueden ocupar una misma franja horaria.

Estado operativo del equipo: Un equipo que se encuentre en mantenimiento, fuera de servicio o inactivo no puede aceptar nuevas reservas.

Control de sesiones: Una sesión de uso solamente puede abrirse a partir de una reserva confirmada y no puede cerrarse más de una vez.

Mantenimiento trazable: Cuando un ticket de mantenimiento se cierre, debe conservarse la información relacionada con la acción realizada y el técnico responsable.

5. Riesgos del Proyecto

Inconsistencia entre UML y código: Existe el riesgo de que el diagrama UML y la implementación Java presenten nombres, tipos, métodos o relaciones diferentes.

Impacto: dificultad para comprender el sistema y pérdida de puntos en la evaluación de coherencia UML-código.

Mitigación: revisar cada clase del UML contra su correspondiente clase Java antes de entregar el proyecto.

Validaciones incompletas: Existe el riesgo de que algunas reglas de negocio se encuentren representadas en el diseño pero no estén completamente implementadas en el código. Por ejemplo, actualmente algunas validaciones están planteadas mediante métodos pero todavía requieren mayor implementación para comprobar todas las condiciones del dominio.

Impacto: reservas inválidas o comportamientos que no cumplen completamente las reglas del proyecto.

Mitigación: probar explícitamente los casos de reserva válida, cruce horario, certificación vencida y equipo no operativo, que son casos exigidos para la Entrega 1.

Errores de integración entre clases: El sistema posee varias relaciones entre usuarios, equipos, reservas, formación, sesiones y mantenimiento.

Impacto: una modificación en una clase puede generar errores en otras partes del sistema.

Mitigación: mantener responsabilidades separadas y probar los flujos completos utilizando objetos relacionados.

6. Decisiones de Modelado

Herencias:

Se utilizo **Usuario** como clase base para **Estudiante, Profesor, Tecnico y AuxiliarLaboratorio**

Especialización:

Equipos se especializa en **Impresora3D, CortadorLaser y Multimetro**, cada uno con características propias.

Formación:

Formacion es la clase base de **Capacitacion y Certificacion**, reutilizando sus atributos comunes.

Reservas y Uso:

Reserva y SesionUso se separaron para diferenciar la programación del uso real del equipo.

Mantenimiento:

TicketMantenimiento permite registrar fallas, prioridad, técnico responsable y cambios realizados.

Polimorfismo:

Se utiliza tipoEspecifico() para que cada tipo de equipo tenga su propio comportamiento.

7. Conclusiones

El modelo desarrollado para MakerLab EIA permite representar los principales elementos involucrados en la administración de laboratorios, usuarios y equipos especializados. La solución utiliza conceptos fundamentales de programación orientada a objetos, especialmente encapsulamiento, herencia, especialización y polimorfismo.

La separación de responsabilidades entre usuarios, equipos, formación, reservas, sesiones de uso y mantenimiento permite construir un modelo que representa de manera organizada el funcionamiento de un MakerLab.

El diseño también establece las bases para futuras ampliaciones del proyecto, como el manejo de colecciones, excepciones personalizadas, patrones de diseño, pruebas unitarias y demás funcionalidades previstas para la Entrega 2.